1.5.2. Achieving new syntax

Deciding on the syntax that you want is a good start. What you need now is some way for the new CoffeeScript syntax to actually work. In order for your CoffeeScript program to execute as a JavaScript program, you need something that can take CoffeeScript,

```
square = (x) ->
  x * x
```

and turn it into the equivalent executable JavaScript:

```
var square = function (x) {
  return x * x;
};
```

That's exactly what a compiler is for, and it's exactly what the CoffeeScript compiler does. On the command line again, if you have a file called square.coffee containing the CoffeeScript, then you convert it to a JavaScript file like so:

```
> coffee –c square.coffee
```

Once that's done, you'll have a new file called square.js containing your newly compiled JavaScript program. That leaves just one question: If the CoffeeScript program must be compiled first, then how does the REPL work? Well, it's read-eval-print, right? The CoffeeScript is simply compiled before evaluation:

```
square = (x) -> x * x          # Read
                               # Compile to JavaScript
                               # Evaluate the resulting JavaScript
# [Function]                   # Print
                               # Loop
square 2                       # Read
                               # Compile to JavaScript
                               # Evaluate the resulting JavaScript
# 4                            # Print
                               # Loop
```